AMC Bank — BFSI Full-Stack Banking Application

8-Day Project Report · Tools & Technologies · Presentation Speech
Java Full-Stack Training Project · September 21–30, 2026

1. The Project in One Paragraph

We built **AMC Bank**, a working internet-banking back office: a **Spring Boot REST API** secured with **JWT login and role-based access** (Admin / Officer / Clerk), backed by a **MySQL** database, and a **React dashboard** that shows live figures — customers, accounts, loans and deposits — with charts. Every endpoint was tested in **Postman**, and the whole stack runs end-to-end on one machine (verified September 30: backend on port 8080, frontend on port 5173).

2. The Work, Day by Day (8 Segments)

Day 1 — Foundation & Environment

Sep 21 · commit "first commit"

Installed and verified the toolchain: JDK, Maven, Node.js, MySQL 8 as a Windows service, Eclipse (Spring Tools), Postman, Git. Created the project repository and the Maven folder layout for the BFSI domain. Outcome: a clean, version-controlled workspace.

Day 2 — Core Java & Spring Fundamentals

Sep 23 · commits "collection and spring core"

Revised collections through banking-flavoured demos — `ArrayList` for passbooks, `HashMap` for balances, `Queue` for cheque processing, `HashSet` to block duplicate PANs — plus equals/hashCode, Comparable/Comparator, generics and the exception family (multi-catch, try-with-resources, custom `BusinessRuleException`). Then Spring Core: IoC and dependency injection — the pattern every later class uses. Outcome: language maturity before framework code.

Day 3 — HTTP, Maven & API Thinking

Sep 24 · commits "html mvn config", "Api Request Response"

Studied the request/response cycle, HTTP methods and status codes, and JSON. Configured `pom.xml` with the Spring Boot parent and starters (web, data-jpa, validation, security). Practised designing API payloads in Postman before writing any controller. Outcome: contract-first thinking.

Day 4 — Database Design & Entities

Sep 25 · commits "Schema", "day04", "react setup"

Designed the MySQL schema: `customer`, `account`, `loan`, `app_user` with proper keys and relationships (one customer → many accounts/loans). Mapped them as JPA entities (`@Entity`, `@OneToMany`, cascade + orphan removal) and created Spring Data repositories. Started the React side with Vite, routing and a layout shell. Outcome: the data layer and the frontend skeleton.

Day 5 — Business Services & Rules

Sep 26 · commits “database”, “pom.xml”, “added statement”

Wrote the service layer: customer CRUD with duplicate-email/PAN guards, account opening with auto-generated numbers (AC9001…), and deposit/withdraw with the insufficient-balance rule. Configured the MySQL connection in `application.properties` and added the reporting query behind the dashboard aggregates. **Outcome: real banking rules enforced in one place.**

Day 6 — React Frontend

Sep 28 · commit “frontend”

Built the UI screens: Login, Dashboard, Customers list/form/detail, Accounts, Loans, Users — using React Router, Axios interceptors (attach the token, handle 401/403 uniformly) and Recharts for the two dashboard charts. **Outcome: a complete, role-aware admin console.**

Day 7 — Security: JWT & Roles

Sep 29 · commits “login”, “day08”, “Update AuthController.java”

The deepest day. Spring Security filter chain, stateless sessions, BCrypt password hashing, `/api/auth/login` issuing JWTs (jjwt 0.12), a request filter validating every token, method-level rules (ADMIN only for deletes and user management; OFFICER for loans), and uniform JSON error responses for 401/403 — all tested in Postman. **Outcome: bank-grade authentication and authorization.**

Day 8 — Seed Data, Integration & E2E Verification

Sep 30 · commit “Add dummy data for customers, accounts, and loans”

Added the `DataSeeder` (three demo logins + ten customers with accounts and loans), wired the non-standard Maven layout (`java/`, `resources/`), fixed the MySQL 8 driver flags, and verified the full flow live: login → JWT → protected endpoints → dashboard charts, plus 401 without a token and 403 for the wrong role. Documented everything in a runbook. **Outcome: a demo-ready, verified product.**

3. Tools & Technologies Used

Tool	Role in the project
Java 17	Core language (compiles to target 17, runs on JDK 25)
Spring Boot 3.2.5	Application framework — REST controllers, dependency injection, auto-configuration
Spring Web (REST)	`@RestController` endpoints: customers, accounts, loans, users, auth, reports
Spring Data JPA / Hibernate	ORM: entities ↔ MySQL tables, repository interfaces, cascade rules
Spring Security + JWT (jjwt)	Stateless login, BCrypt hashing, token filter, role-based access (ADMIN/OFFICER/USER)
Hibernate Validator	`@Valid` request-body validation with field-level error messages
MySQL 8 + Connector/J	Persistent storage (schema `bfsi_db`, auto-created on boot)
H2 (in-memory)	Zero-setup test/demo database profile
Maven 3.9	Build tool, dependency management, `spring-boot:run`
React 18 + Vite 5	Frontend single-page app with hot reload dev server
React Router 6	Client-side navigation; protected routes per role
Axios	HTTP client: token interceptor and unified error handling
Recharts	Dashboard bar charts (loans by type, deposits by city)
Postman	API testing: every method, role and error case
Eclipse + Spring Tools	IDE for backend development and debugging
Node.js + npm	Frontend runtime and package management
Git / GitHub	Version control — one commit-set per work day
curl	Scripted smoke tests during integration

4. Presentation Speech (≈ 4 minutes)

> *Good morning everyone. Over the last eight days I built **AMC Bank** — a full-stack banking application — and I'd like to walk you through what we did, the tools we used, and what I learned.*
>
> ***Where we started.** Day one wasn't code — it was foundations. I installed and verified the entire toolchain: Java, Maven, MySQL as a Windows service, Eclipse with Spring Tools, Postman and Git. From the very first commit, every day's work is visible in version control — a discipline I kept all through the project.*
>
> ***Core Java first.** Before touching Spring, I revised collections the practical way: an ArrayList as a passbook, a HashMap for balances, a HashSet to catch duplicate PANs, a Queue for cheque processing — plus generics and the full exception family. That paid off immediately, because banking code is mostly data structures and edge cases.*
>
> ***Then the API.** We designed the REST contract in Postman before writing controllers — request, response, status codes. The backend is Spring Boot with three layers: controllers that speak HTTP, services that enforce the business rules — like blocking a withdrawal that overdraws an account — and Spring Data repositories that talk to MySQL through JPA entities. One customer can hold many accounts and loans, handled with proper cascades.*
>
> ***Security was the deepest day.** The API is stateless and token-based: you log in, Spring Security verifies BCrypt-hashed passwords and issues a JWT; every request passes through a filter that validates the token. Access is role-based — Admin can delete and manage users, Officers sanction loans, Clerks can read and create but not delete. Wrong password gets a clean 401; right token but wrong role gets a 403 — all in a uniform JSON error shape the frontend understands.*
>
> ***The frontend** is a React single-page app built with Vite: a login screen, a dashboard with live figures and two charts, and full screens for customers, accounts, loans and users. An Axios interceptor attaches the token to every call and reacts automatically when a session expires.*
>
> ***How it was verified.** Every endpoint was tested in Postman — happy paths and error paths. On the final day I integrated everything end-to-end on real MySQL: seeded demo data, logged in through the UI, watched the dashboard pull live numbers — eleven customers, fourteen accounts, nine active loans — and confirmed the security boundaries behave exactly as designed.*
>
> ***What I take away:** how the pieces actually fit — HTTP → controller → service → repository → database and back; why security belongs in a filter chain, not sprinkled through the code; and how a frontend and backend negotiate through a clear API contract. Everything you saw today is running live on this machine — thank you, and I'm happy to take questions.*